Session 10

CSS Layout & Interaction – Box Model, Display, and Pseudo-Classes

What are you going to learn in this session?

In this session, you will learn how CSS controls spacing, layout behavior, and simple user interaction on a web page. Until now, you have styled content using colors, fonts, and basic selectors. In this class, you will begin shaping how elements sit on the page and how they respond when users interact with them.

You will understand the box model, width and height, display behavior, visibility differences, basic specificity, and pseudo-classes such as `:hover`, `:active`, and `:focus`. By the end of the session, you will improve your existing project so it feels cleaner, better spaced, and more interactive.

Why is this topic important?

A page can have good colors and typography but still look messy if spacing and layout are not handled properly. Most beginner websites look crowded or awkward not because of bad content, but because the spacing between elements is poor and the layout behavior is not understood.

This session fixes that problem by teaching you how browsers treat every element as a box and how CSS lets you control the space inside and outside that box. It also introduces interaction states, which are essential for giving users visual feedback when they click buttons, focus on fields, or move their mouse over elements.

How should you prepare yourself to attend this session?

Before attending this session, make sure your existing HTML and CSS project is open and working properly in VS Code with Live Server. You should already be comfortable linking your CSS file, writing basic selectors, and applying simple styles.

Come prepared to experiment visually. This session becomes much easier when you actively change values, refresh the page, and observe what happens. Keep your browser DevTools ready as well, because seeing the box model visually makes the concept much easier to understand.

What should you do after the session?

After the session, you should revisit every major section of your project and improve spacing deliberately instead of randomly. Add margin where elements need breathing room, add padding where content feels too close to the border, and use borders where structure is unclear.

You should also practice adding hover and focus effects to common interactive elements such as buttons, links, and form fields. The goal is to develop visual judgment along with CSS syntax, because strong frontend work depends on both.

Class Notes

Box Model (Core Concept)

In CSS, every HTML element is treated like a box. This is one of the most important ideas in frontend development because layout, spacing, and sizing all depend on it. Once you understand the box model, many CSS behaviors start making sense.

The box model has four parts: content, padding, border, and margin. Content is the actual text or image inside the element. Padding is the inner space between the content and the border. Border is the visible boundary around the element. Margin is the outer space that separates the element from other elements.

A simple example looks like this:

```CSS
.card {
  padding: 20px;
  border: 2px solid #333;
  margin: 16px;
}
```

If you apply this class to a section or card, you will immediately notice the difference between internal spacing and external spacing. Padding pushes content inward from the border, while margin creates distance outside the element.

Width and Height

Every element has a default sizing behavior based on its content and display type. CSS allows you to control width and height, which is useful when you want sections, images, or cards to occupy a certain amount of space.

For example, you may want an image to be smaller than its original size or a content section to occupy only a portion of the page width. At the same time, you should be careful with fixed heights because content can overflow or get cut off when the amount of content changes.

```CSS
.hero-section {
  width: 80%;
}

.banner-image {
  width: 300px;
}

.box {
  height: 150px;
}
```

In practice, width is used frequently to control layout, while height should be used carefully unless you are very sure about the content inside the element.

Display Property

Different HTML elements behave differently by default. Some elements take up the full available width, while others occupy only as much space as their content needs. The `display` property allows you to control this behavior.

Block elements take full width and start on a new line. Inline elements only take up as much width as their content and stay in the flow of text. Inline-block elements are a useful middle ground because they stay inline but still allow width, height, margin, and padding to be applied more predictably.

```CSS
h1 {
  display: block;
}

span {
  display: inline;
}

.button-like-link {
```

```
  display: inline-block;
  padding: 10px 16px;
}
```

This becomes very useful when styling navigation links, cards, badges, and buttons, because you can choose how much space they occupy and how they align with nearby elements.

Visibility vs Display None

Sometimes you want to hide an element, but the way you hide it affects the page layout. This is where `display: none` and `visibility: hidden` behave differently.

When you use `display: none`, the element is removed completely from the layout as if it never existed. When you use `visibility: hidden`, the element becomes invisible but still occupies its original space on the page.

```CSS
.hidden-box {
  display: none;
}

.invisible-box {
  visibility: hidden;
}
```

This difference matters when designing interfaces, because hiding an element may either collapse the layout or leave an empty gap depending on which property you use.

Basic Specificity

Specificity explains why one CSS rule sometimes overrides another. Beginners often feel confused when a style does not apply, and in many cases the reason is not a syntax error but a stronger competing selector.

At a simple level, you can remember the priority like this: inline styles are strongest, then id selectors, then class selectors, and then tag selectors. This is enough for beginner debugging and will help you understand why some styles win over others.

HTML

```
<h1 id="title" class="heading" style="color: red;">Welcome</h1>
```

CSS

```
h1 {
  color: blue;
}

.heading {
  color: green;
}

#title {
  color: purple;
}
```

In this case, the final color will be red because the inline style has the highest priority. You do not need deep theory yet, but you should begin building the habit of checking selector strength when styles behave unexpectedly.

Pseudo-Classes (Interaction)

Pseudo-classes allow you to style elements based on user interaction or element state. This is what makes interfaces feel alive instead of static. Even a small hover effect can make a page feel much more polished.

The most common beginner pseudo-classes are `:hover`, `:active`, and `:focus`. Hover applies when the mouse is over an element, active applies while an element is being clicked, and focus applies when an input or interactive element is selected.

CSS

```
button:hover {
  background-color: #333;
  color: white;
}

button:active {
  transform: scale(0.98);
}
```

```
input:focus {
  border: 2px solid blue;
  outline: none;
}
```

These states are especially useful for buttons, links, and form fields because they give users immediate feedback and improve usability.

Applying Styling to the Project

Now you will use all of these concepts to improve your existing project in a practical way. Focus first on the header, table, and form, because these areas usually show visible improvement quickly when spacing and structure are handled well.

You can begin by adding classes to important sections and then applying margin, padding, and borders to create better visual separation. After that, adjust display behavior where needed, and finally add hover and focus states to interactive elements.

A small example may look like this:

```CSS
header {
  padding: 20px;
  margin-bottom: 20px;
  border-bottom: 2px solid #ddd;
}

form input {
  display: block;
  margin-bottom: 12px;
  padding: 10px;
}

a:hover {
  color: darkred;
}
```

The aim is not perfection. The aim is to make the page visibly cleaner, easier to use, and more pleasant to interact with.

Common Beginner Confusions

Many learners confuse margin and padding because both create space, but they do it in different places. Padding adds space inside the element around the content, while margin adds space outside the element between it and other elements.

Another common issue is display behavior not matching expectations. For example, learners often try to give width to an inline element and then wonder why it does not behave as expected. In many cases, changing the display value solves the problem.

Pseudo-classes may also seem like they are not working, especially if the selector is wrong or the element is not actually interactive in the way you expect. Similarly, if CSS is not applying at all, specificity conflicts or selector mismatches are often the real cause.

Key Takeaways

You now understand that every HTML element behaves like a box, and that CSS gives you control over the content area, padding, border, and margin. This allows you to manage spacing and structure much more intentionally instead of adjusting styles randomly.

You have also learned how width, height, and display affect layout, how `display: none` differs from `visibility: hidden`, and how pseudo-classes help create basic interactivity. These concepts are central to making pages feel structured, readable, and responsive to user action.

At this stage, your progress will come from hands-on experimentation. The more you inspect, adjust, and compare visual results, the stronger your CSS intuition will become.